ISYE 7406 HW5

Introduction

This analysis will explore the applications of Random Forest and GBM Boosting for my course project focused on investigating healthcare and socioeconomic factors that influence life expectancy, and compare these methods to common baseline methods. The data mining challenges to consider are the sophistication of panel data (various violations of I.I.D. and technique usage, potential issues with boosting computation) and collinearity of certain variables. These issues can be tackled by introducing transformations, non-linear approaches, and by conducting EDA to determine the severity of multicollinearity if it exists.

Data Source

https://www.kaggle.com/datasets/lashagoch/life-expectancy-who-updated
This dataset is obtained from Kaggle–named "Life Expectancy WHO (Updated)". Each of the factors in the dataset are sourced from certain institutions:

- World Health Organization (WHO): Life expectancy; mortality rates per 1000; alcohol consumption; immunizations for Hepatitis B, Measles, Polio, and DTP3; BMI; HIV; and thinness.
- World Bank: GDP per capita, population.
- Our World In Data: Years of schooling.

The dataset consists of 21 variables and 2864 observations (179 countries x 16 years). However, this changes with the data wrangling step that is explored in the Exploratory Data Analysis section. Furthermore, the dataset builds upon the original Kaggle dataset from 5 years prior to this dataset's creation called Life Expectancy WHO (https://www.kaggle.com/datasets/kumarajarshi/life-expectancy-who), but the key difference is that the newer and updated Kaggle dataset I am using includes imputation methods to address missing values and some suspicious outliers from the original dataset (data from certain countries are not easily accessible). Thus, while it is important to consider the validity of the dataset, the author of the dataset reasonably justifies the imputation:

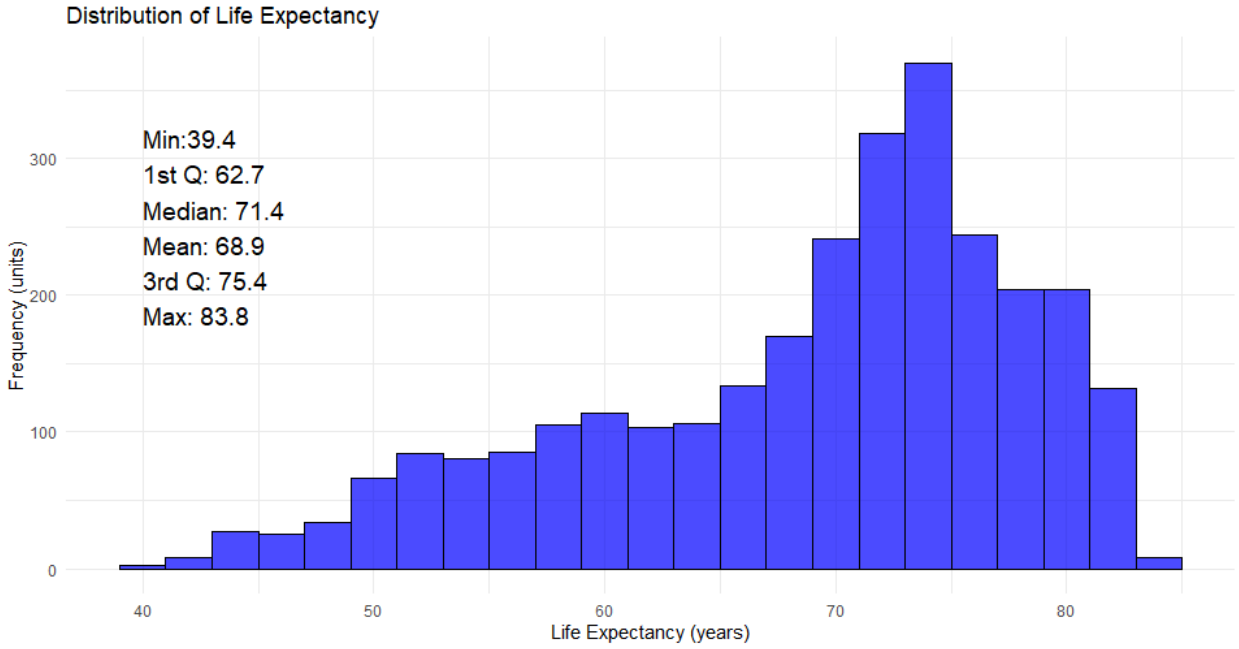
- For a country's missing values in any year: the closest three-year average.
- For a country's missing values in all years: average of the Region.
- Certain countries with more than 4 missing columns were omitted.
    - Down from 193 countries in the original dataset to 179 countries in the current.
- The original kaggle dataset had also utilized Missmap in R for imputation to address missing data for smaller countries such as Togo and Cabo Verde.

Again, it is important to note the I.I.D. violations throughout the entire dataset, which affects the application of the ML techniques used in this homework and the project. Much of the focus on the methodology will be to minimize such violations.

Exploratory Data Analysis

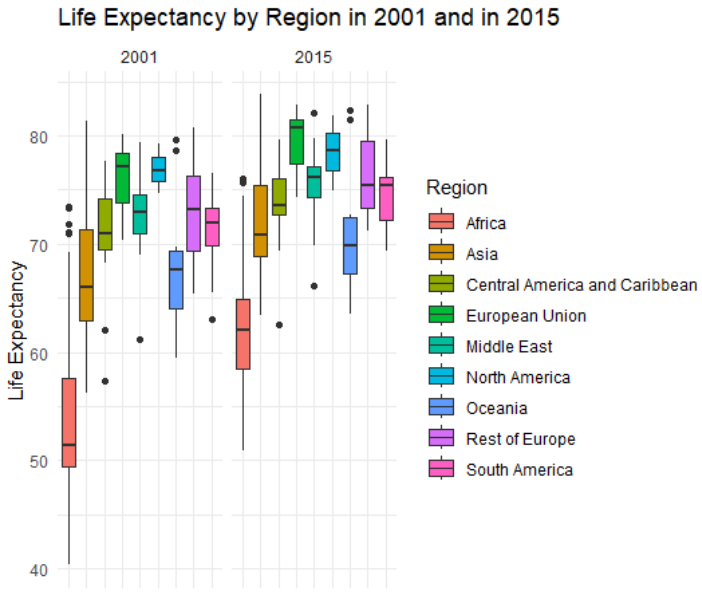
A number of visualizations are created below to provide some insights into the dataset.

1) Distribution of Life Expectancy

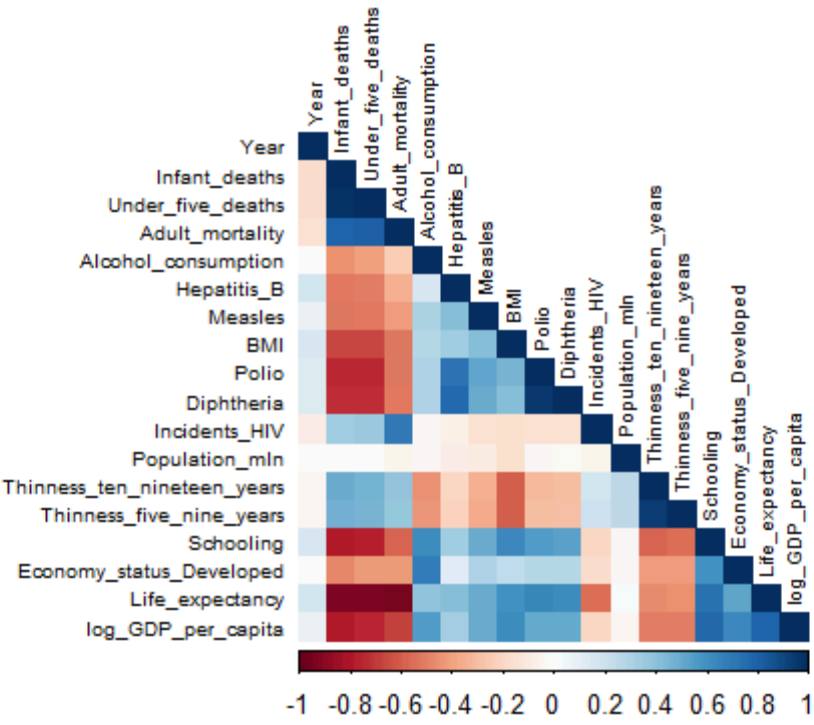


The distribution of life expectancy is right-skewed, with the majority of instances falling between 70 to 80 years. This also implies that techniques that assume a normal distribution will struggle.

2)



The disparities of life expectancy in 2001 were much larger than in 2015, and there is an overall increase in life expectancy throughout the years for all regions concerned. It may be useful to include a regional classification as a feature in model building.



3)

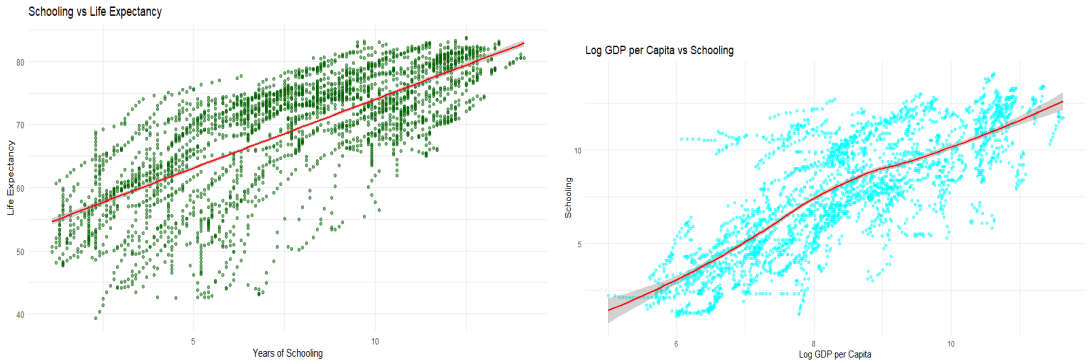
In the correlation matrix, there are some notable correlations against life expectancy:

- Strong positive correlations include schooling and GDP per capita.
- Strong negative correlations include all the mortality factors (infant, under 5, and adult)
- Weaker correlations include vaccination rates and economy status

To address multicollinearity concerns, AIC will be applied for feature selection, particularly to address the correlations between GDP per capita vs. schooling, and the mortality rates.

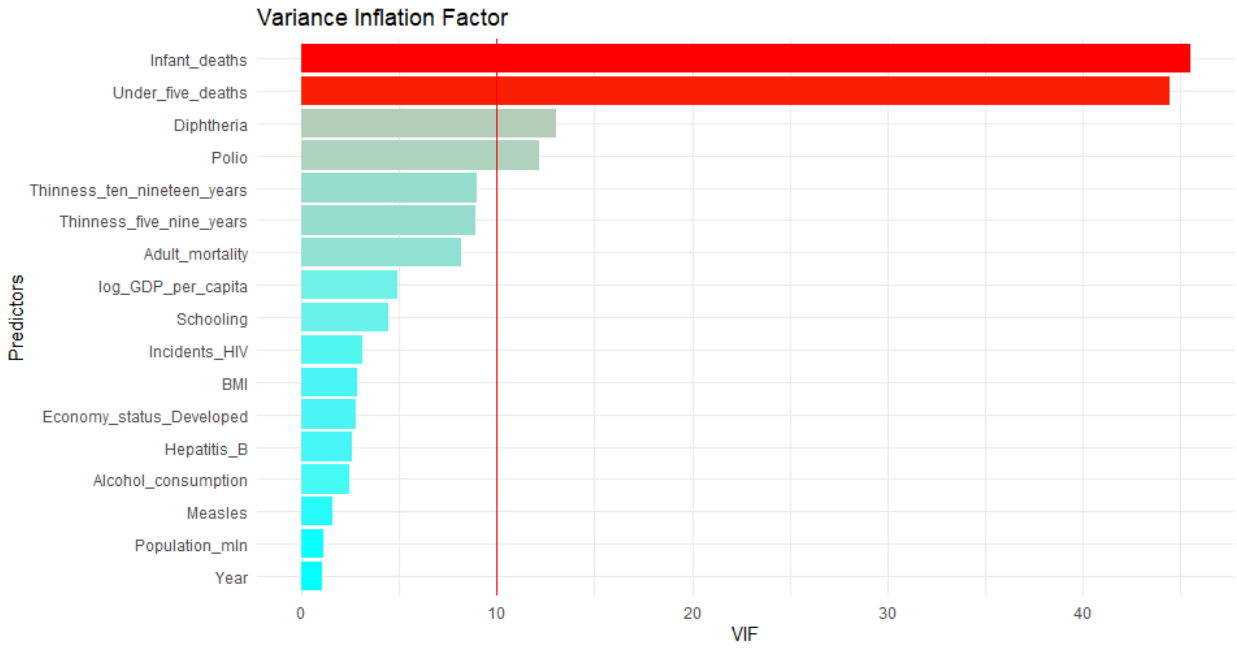
4)

There is a strong nonlinear relationship between GDP per capita and life expectancy. In particular, it shows diminishing returns of life expectancy at higher GDP levels. It makes sense to implement a log transformation on GDP per capita, as the trendline differences (using LOESS) indicates that logGDP per capita appears to be much more linear. I also considered transformations for other variables, but the justification for transformations is not the same in every case (e.g. hinders interpretability). In addition, Random Forest and Boosting do not assume linearity, unlike certain baseline models.



5)

Based on the green plot, years of schooling and life expectancy are strongly positively correlated, and by plotting log GDP per capita against years of schooling in the cyan plot, I immediately thought to test for multicollinearity.



6)

After testing for multicollinearity using VIF, the VIF values are below 5, suggesting that there is some correlation between log GDP per capita and schooling, but it is not serious. In addition, this led me to investigate other variables for multicollinearity. 4 variables were above 10–a loosely defined threshold to indicate severe multicollinearity. Of those four variables, two of them have a value of around 45 (infant deaths per 1,000) and 44 (under five deaths per 1,000). This is extremely undesirable and unsurprising, and I opted to remove infant deaths because under five deaths include infant deaths. Diphtheria (DTP3) and Polio (both represented by the percent coverage of immunization among 1 year olds) have high factors and can be attributed to the likelihood of having coverage of both vaccines. In other words, one may assume that if a country has a high DTP3 vaccination rate, then it likely has high Polio vaccination rates as well. Therefore, Polio is also removed from model considerations as DTP3 is often used as an immunization benchmark (UNICEF, 2024).

Methodology

The two main methods are Random Forests and GBM Boosting. They will be compared to baseline methods of linear regression, stepwise AIC, KNN, and LOESS smoothing. Using AIC stepwise for feature selection, all models (with the exception of the linear regression with the training data at the start) will utilize the same set of features.

For cross-validation, it will not utilize a random train-test split but instead a time-aware train-test split of 80/20 due to the longitudinal nature of the dataset. Therefore, the train-test split will be created based on data from the years 2001 to 2012 (training data), and tested on 2013 to 2015, which resembles real life efforts of predicting future years. The main and baseline methods will be compared by the testing error in a single run evaluation, followed by cross-validation of 100 iterations.

1. Linear Regression

Linear Regression is the first baseline and serves as a benchmark for the succeeding models because of its simplicity. It is expected to struggle with the existence of nonlinear relationships in the dataset.

2. Regression using AIC Stepwise

Feature selection is implemented through minimizing AIC for linear regression. Population (in millions), Measles, and Thinness from 5-9 years were removed to minimize RSS using the stepwise function, which helps to strike a balance between simplifying models and maintaining predictive power.

3. KNN

KNN is utilized because it is a simpler method of tackling nonlinear patterns and classification.The optimal k was found to be k = 1, but because of overfitting and is not very meaningful, k = 3 is used instead, as the jump in testing error is very minor.

K	Testing Error	K	Testing Error
1	1.058901	9	1.557114
2	1.063841	10	1.595940
3	1.174558	11	1.611758
4	1.267886	12	1.684490
5	1.339035	13	1.713855
6	1.407628	14	1.776255
7	1.444596	15	1.791970
8	1.505638		

4. LOESS

LOESS is limited to 4 predictors in R and is low-dimensional in nature. It is a smoother that allows for the fitting of many regressions around different points in a dataset to capture complexities beyond the linear scale. Since I have already transformed GDP per capita, I decided on other nonlinear predictors: Alcohol consumption, under five deaths, adult_mortality, and BMI. A sequence of 0.1 to 1 by every 0.05 intervals was tested to find the optimal span parameter. This is found to be 0.9, where the testing error is at its minimum.

5. Random Forest

Random Forest (and GBM Boosting) are selected because of its ability to handle nonlinear relationships–something that is highly prevalent in this dataset. Due to computational cost, the parameters were tested for within a sparing range:

- ntree: 100, 200, 400, 600, 800. Optimal: 400
- nodesize: 1, 3, 5, 7, 9. Optimal: 1
- Mtry: 1, 3, 5, 7, 9, 11, 13, 15. Optimal: 5

6. GBM Boosting

With the same computational issues that Random Forest struggles with, the parameters to consider are as follows:

- n.trees: 100, 200, 400, 800, 1600. Optimal: 1600 (could be overfitting)
- shrinkage: 0.01 - 0.1 by increments of 0.01. Optimal: 0.1
- interaction.depth: 1, 3, 5. Optimal: 3
- n.minobsinnode: 5, 10, 15. Optimal: 5

Results

In the single run scenarios, the testing errors are as listed:

Method	Testing Error
Linear Regression	1.951338
Regression Using AIC	1.950082
KNN (k = 3)	1.174558
LOESS	1.579921
Random Forest	0.699868
GBM Boosting	0.584629

Based on the table of results above, the best performing models were GBM Boosting and Random Forest by far. This is followed by KNN at k=3, then followed by LOESS, and finally with the traditional regression methods performing the worst, which suggests that the dataset requires complex tools to tackle the abnormal relationships and distributions. Parameters were largely tuned to optimal settings and to the best of the computer's ability, as it became increasingly difficult to compute the optimal parameters for Random Forest and GBM Boosting.

Using Monte-Carlo Cross-Validation of 100 iterations, the testing error results are as follows:

Method	Average Testing Error	Variance
Linear Regression	1.854161	0.010781
Regression Using AIC	1.853999	0.010918
KNN (k = 3)	0.494287	0.005200
LOESS	1.346724	0.006781
Random Forest	0.226755	0.000795
GBM Boosting	0.285237	0.002360

These results seem to indicate that Random Forest and GBM Boosting are the strongest performers, with Random Forest as the best performer this time around. Their average testing error drastically decreases. KNN remains as the third lowest average testing error, but it also witnessed the largest decrease in testing error when compared to the single run. The order largely remains the same beyond the changes in the order of the main methods. Thus, the cross-validation helps to reassert that Random Forest and GBM Boosting are the strongest performers for the life expectancy dataset.

Conclusions

This analysis investigated the effectiveness of Random Forest and GBM Boosting in predicting life expectancy based on healthcare and socioeconomic factors. By comparing these two methods to baseline methods, it became increasingly clear that the baseline methods struggle to capture the sophistication of the panel data. Conversely, the main methods excelled at predicting testing data. In addition, the Monte-Carlo Cross-Validation bolstered these findings, showcasing the widening gap between the performance of tree-based models versus the simple, linear techniques. Thus, the results indicate a need for sophisticated techniques for learning with nonlinear and time-dependent data.

Lessons I have learned

This homework was very beneficial for the preparation of my final project. It helped me to understand preprocessing and tackling the inherent issues with my panel data, such as how it did not seem ideal to just randomly split training and testing data on longitudinal datasets. I learned a lot about the computational difficulties regarding tree-based models, when most techniques I have learned in the past were much more straightforward and easier to work with. I feel more confident in managing non-linear relationships and how to avoid getting too comfortable with the simpler baseline methods for such complex data, but I also need to consider lagged variables and timing adjustments given the nature of the dataset. In addition, I hope to gain more knowledge from the final module on unsupervised learning in order to explore how one might utilize clustering techniques to categorize countries into economic development labels.

Appendix

```
library(ggplot2)
library(GGally)
library(readr)
library(dplyr)
library(caret)
library(tidyr)
library(ggcorrplot)
library(corrplot)
library(MASS)
library(car)
library(splines)
library(randomForest)
library(gbm)

set.seed(100)
# Dataset
LE <- read_csv("Life-Expectancy-Data-Updated.csv")

LEdata <- LE %>%
  arrange(Country, Year) %>%
  mutate(log_GDP_per_capita = log(GDP_per_capita + 1)) %>% # Transformation
  dplyr::select(-Economy_status_Developing) # Redundant dummy

# EDA
summary(LEdata)

# Generate Histogram with Summary Table
summary_stats <- quantile(LEdata$Life_expectancy, probs = c(0.25, 0.50, 0.75))
summary_text <- paste0(
  "Min:", round(min(LEdata$Life_expectancy), 1), "\n",
  "1st Q: ", round(summary_stats[1], 1), "\n",
  "Median: ", round(summary_stats[2], 1), "\n",
  "Mean: ", round(mean(LEdata$Life_expectancy), 1), "\n",
  "3rd Q: ", round(summary_stats[3], 1), "\n",
  "Max: ", round(max(LEdata$Life_expectancy), 1)
)
ggplot(LEdata, aes(x = Life_expectancy)) +
  geom_histogram(binwidth = 2, fill = "blue", color = "black", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Distribution of Life Expectancy", x = "Life Expectancy
(years)", y = "Frequency (units)") +
  annotate("text", x = 40, y = 250, label = summary_text, hjust = 0, size = 5,
color = "black")

# Boxplots of Life Expectancy by Region for 2001 and 2015
LEdata_filtered <- LEdata %>%
  filter(Year %in% c(2001, 2015))
```

```r
ggplot(LEdata_filtered, aes(x = Region, y = Life_expectancy, fill = Region)) +
  geom_boxplot() +
  facet_wrap(~Year) +
  theme_minimal() +
  theme(axis.text.x = element_blank(),
        axis.ticks.x = element_blank()) +
  labs(title = "Life Expectancy by Region in 2001 and in 2015", x = NULL, y =
"Life Expectancy")

# GDP per capita and Log-GDP per capita vs Life Expectancy
ggplot(LEdata, aes(x = GDP_per_capita, y = Life_expectancy)) +
  geom_point(alpha = 0.5, color = "blue") +
  geom_smooth(method = "loess", color = "red") +
  theme_minimal() +
  labs(title = "GDP per Capita vs Life Expectancy", x = "GDP per Capita", y =
"Life Expectancy")
ggplot(LEdata, aes(x = log_GDP_per_capita, y = Life_expectancy)) +
  geom_point(alpha = 0.5, color = "blue") +
  geom_smooth(method = "loess", color = "red") +
  theme_minimal() +
  labs(title = "Log GDP per Capita vs Life Expectancy", x = "Log GDP per
Capita", y = "Life Expectancy")

# Schooling vs Life Expectancy
ggplot(LEdata, aes(x = Schooling, y = Life_expectancy)) +
  geom_point(alpha = 0.5, color = "darkgreen") +
  geom_smooth(method = "lm", color = "red") +
  theme_minimal() +
  labs(title = "Schooling vs Life Expectancy", x = "Years of Schooling", y =
"Life Expectancy")

# Log-GDP vs Schooling
ggplot(LEdata, aes(x = log_GDP_per_capita, y = Schooling)) +
  geom_point(alpha = 0.5, color = "cyan") +
  geom_smooth(method = "loess", color = "red") +
  theme_minimal() +
  labs(title = "Log GDP per Capita vs Schooling", x = "Log GDP per Capita", y =
"Schooling")

# Correlation matrix
LEdata <- LEdata %>%
  dplyr::select(-GDP_per_capita)
LEdata_numeric <- LEdata %>%
  dplyr::select(where(is.numeric))
cor_matrix <- cor(LEdata_numeric, use = "everything")
corrplot(cor_matrix, method = "color", type = "lower", tl.cex = 0.6, tl.col =
"black")

# VIF check and plot
```

```r
vif_values <- vif(lm(Life_expectancy~., data=LEdata_numeric))
vif_df <- data.frame(Variable = names(vif_values), VIF = vif_values)
vif_df <- vif_df %>% arrange(desc(VIF))
ggplot(vif_df, aes(x = reorder(Variable, VIF), y = VIF, fill = VIF)) +
  geom_bar(stat = "identity") +
  coord_flip() +  # Flip for better readability
  theme_minimal() +
  labs(title = "Variance Inflation Factor",
       x = "Predictors",
       y = "VIF") +
  scale_fill_gradient(low = "cyan", high = "red") +
  geom_hline(yintercept = 10, linetype = "solid", color = "red") +
  theme(legend.position = "none")

LEdata_numeric <- LEdata_numeric %>%
  dplyr::select(where(is.numeric)) %>%
  dplyr::select(-Infant_deaths, -Polio)
vif(lm(Life_expectancy~., data=LEdata_numeric))

LEdata <- LEdata %>%
  dplyr::select(-Infant_deaths, -Polio)

# Split dataset into training and testing
train_data <- LEdata_numeric %>% filter(Year >= 2001 & Year <= 2012)
test_data  <- LEdata_numeric %>% filter(Year >= 2013 & Year <= 2015)

# 1. Linear Regression
linreg_model <- lm(Life_expectancy ~ ., data=train_data)
pred1 <- predict(linreg_model, test_data)
te1 <- mean((pred1 - test_data$Life_expectancy)^2)
te1

# 2. Regression using AIC
aic_filter <- stepAIC(linreg_model, direction = "both", trace = TRUE)
aic_model <- lm(Life_expectancy ~ Year + Under_five_deaths + Adult_mortality +
                  Alcohol_consumption + Hepatitis_B + BMI + Diphtheria +
Incidents_HIV +
                  Thinness_ten_nineteen_years + Schooling +
Economy_status_Developed +
                  log_GDP_per_capita, data=train_data)
pred2 <- predict(aic_model, test_data)
te2 <- mean((pred2 - test_data$Life_expectancy)^2)
te2

# 3. KNN - finding optimal k
k_test <- numeric(15)
for (i in 1:15) {
  knn_model <- knnreg(train_data %>% dplyr::select(-Life_expectancy),
                      train_data$Life_expectancy,
```

```
                      i)
  knn_pred <- predict(knn_model, test_data %>% dplyr::select(-Life_expectancy))
  k_test[i] <- mean((knn_pred - test_data$Life_expectancy)^2)
}
print(k_test)
# k = 1 is optimal, but opting to use k = 3
knn_model <- knnreg(train_data %>% dplyr::select(-Life_expectancy),
                    train_data$Life_expectancy,
                    k = 3)
pred3 <- predict(knn_model, test_data %>% dplyr::select(-Life_expectancy))
te3 <- mean((pred3 - test_data$Life_expectancy)^2)
te3

# 4. LOESS
# Finding optimal span
span_range <- seq(0.1, 1, 0.05)
loess_errors <- numeric(length(span_range))
for (i in seq_along(span_range)) {
  loess_optimal_span <- loess(Life_expectancy ~ Adult_mortality +
Under_five_deaths +
                        BMI + Alcohol_consumption,
                      data = train_data,
                      span = span_range[i],
                      degree = 2)
  loess_pred <- predict(loess_optimal_span, newdata = test_data)
  loess_errors[i] <- mean((loess_pred - test_data$Life_expectancy)^2, na.rm =
TRUE)
}
optimal_span <- span_range[which.min(loess_errors)]
# Optimal span is 0.9. Solve for lowest testing error
loess_model <- loess(Life_expectancy ~ Adult_mortality + Under_five_deaths +
                       BMI + Alcohol_consumption,
                     data = train_data,
                     span = optimal_span,
                     degree = 2)
pred4 <- predict(loess_model, newdata = test_data)
te4 <- mean((pred4 - test_data$Life_expectancy)^2, na.rm = TRUE)
te4

# 5. Random Forest
# Optimize ntree, nodesize, and mtry

# Load required package
library(randomForest)

# Define parameter grid
ntree_value <- c(100, 200, 400, 600, 800)
nodesize_value <- seq(1, 9, 2)
mtry_value <- seq(1, 15, 2)
```

```r
# Store parameter values in grid
rf_grid <- expand.grid(ntree = ntree_value,
                       mtry = mtry_value,
                       nodesize = nodesize_value)
rf_grid$Test_Error <- NA

# Test for optimal parameters (long search)
for (i in 1:nrow(rf_grid)) {
  rf_optimal_param <- randomForest(Life_expectancy ~ .,
                           data = train_data,
                           ntree = rf_grid$ntree[i],
                           mtry = rf_grid$mtry[i],
                           nodesize = rf_grid$nodesize[i])

  rf_pred <- predict(rf_optimal_param, test_data)
  rf_grid$Test_Error[i] <- mean((rf_pred - test_data$Life_expectancy)^2)
}

optimal_rf <- rf_grid[which.min(rf_grid$Test_Error), ]
print(optimal_rf)
# Apply randomforest using optimal parameters
rf_model <- randomForest(Life_expectancy ~ .,
                         data = train_data,
                         ntree = 400,
                         mtry = 5,
                         nodesize = 1)
pred5 <- predict(rf_model, test_data)
te5 <- mean((pred5 - test_data$Life_expectancy)^2)
te5

# 6. GBM
gbm_grid <- expand.grid(
  interaction.depth = c(1, 3, 5),
  n.trees = c(100, 200, 400, 800, 1600),
  shrinkage = seq(0.01, 0.1, 0.01),
  n.minobsinnode = c(5, 10, 15)
)
gbm_results <- data.frame()

for (i in 1:nrow(gbm_grid)) {
  optimal_gbm_param <- gbm(
    Life_expectancy ~ .,train_data,
    distribution = "gaussian",
    n.trees = gbm_grid$n.trees[i],
    interaction.depth = gbm_grid$interaction.depth[i],
    shrinkage = gbm_grid$shrinkage[i],
    n.minobsinnode = gbm_grid$n.minobsinnode[i],
    cv.folds = 1,
```

```
    verbose = FALSE
  )
}
# Best parameters are found
best_gbm <- gbm_results[which.min(gbm_results$MSE), ]

gbm_model <- gbm(
  formula = Life_expectancy ~ .,
  data = train_data,
  distribution = "gaussian",
  n.trees = 1600,
  interaction.depth = 3,
  shrinkage = 0.1,
  n.minobsinnode = 5,
  verbose = FALSE
)

pred6 <- predict(gbm_model, test_data, n.trees = 1600)
te6 <- mean((pred6 - test_data$Life_expectancy)^2)
te6

# MCCV 100 Iterations
m = 100
test_errors <- matrix(NA, nrow = m, ncol = 7)
colnames(test_errors) <- c("Linear Regression", "AIC Regression", "KNN",
"LOESS", "Spline", "Random Forest", "GBM Boosting")

for (i in 1:m) {
  cat("Iteration:", i, "\n")

  # Random 80/20 split
  train_index <- createDataPartition(LEdata_numeric$Life_expectancy, p = 0.8,
list = FALSE)
  train_data <- LEdata_numeric[train_index, ]
  test_data <- LEdata_numeric[-train_index, ]

  ### 1. Linear Regression
  linreg_model <- lm(Life_expectancy ~ ., data = train_data)
  pred1 <- predict(linreg_model, test_data)
  test_errors[i, 1] <- mean((pred1 - test_data$Life_expectancy)^2)

  ### 2. AIC Regression
  aic_model <- stepAIC(linreg_model, direction = "both", trace = FALSE)
  pred2 <- predict(aic_model, test_data)
  test_errors[i, 2] <- mean((pred2 - test_data$Life_expectancy)^2)

  ### 3. KNN (k = 3)
  knn_model <- knnreg(train_data %>% dplyr::select(-Life_expectancy),
train_data$Life_expectancy, k = 3)
```

```r
  knn_pred <- predict(knn_model, test_data %>% dplyr::select(-Life_expectancy))
  test_errors[i, 3] <- mean((knn_pred - test_data$Life_expectancy)^2)

  ### 4. LOESS
  loess_model <- loess(Life_expectancy ~ Adult_mortality + Under_five_deaths +
BMI + Alcohol_consumption,
                       data = train_data, span = 0.9, degree = 2)
  loess_pred <- predict(loess_model, newdata = test_data)
  test_errors[i, 4] <- mean((loess_pred - test_data$Life_expectancy)^2, na.rm =
TRUE)

  ### 5. Random Forest
  rf_model <- randomForest(Life_expectancy ~ ., data = train_data, ntree = 400,
mtry = 5, nodesize = 1)
  rf_pred <- predict(rf_model, test_data)
  test_errors[i, 5] <- mean((rf_pred - test_data$Life_expectancy)^2)

  ### 6. GBM Boosting
  gbm_model <- gbm(Life_expectancy ~ ., data = train_data, distribution =
"gaussian",
                   n.trees = 1600, interaction.depth = 3, shrinkage = 0.1,
n.minobsinnode = 5, verbose = FALSE)
  gbm_pred <- predict(gbm_model, test_data, n.trees = 1600)
  test_errors[i, 6] <- mean((gbm_pred - test_data$Life_expectancy)^2)
}
te_avg <- colMeans(as.data.frame(test_errors))
te_avg
te_var <- apply(test_errors, 2, var)
te_var
```

Bibliography

In Focus: Immunization. UNICEF Europe and Central Asia. (2024, November 1). https://www.unicef.org/eca/reports/focus-immunization-2024